Modelo jerárquico espacial — ATUS ZMM

El paso 5 de la escalera: un modelo bayesiano jerárquico de conteos para las celdas de la Zona Metropolitana de Monterrey, con la especificación que decidió `modelo_conteo.qmd` — Binomial Negativa, municipio como nivel con efecto que cambia en el tiempo, y efecto espacial por celda para el *shrinkage*. Se ajusta por MCMC (NUTS), se diagnostica la convergencia, se valida en 2024 contra los *baselines* y se prueba la sensibilidad a los priors.

Fuente: `data/processed/atus_zmm_limpio_geo.parquet`, particionado con `geostats.espacial.celdas`.

```
uv run limpiar-atus --geo
```

Qué hay aquí

§	Tema
1	El panel anual, y por qué anual
2	El modelo: ecuación, jerarquía, priors y supuestos
3	Por qué CAR propio y no BYM2
4	Ajuste y convergencia
5	Lo que el modelo aprendió: parámetros y trayectorias municipales
6	El perfil espacial de las celdas: *shrinkage* y mapa
7	Validación en 2024
8	Sensibilidad a los priors

> **Advertencia que atraviesa todo el documento.** El nivel municipal se modela con innovaciones anuales de colas pesadas (*t* de Student), porque así son los datos. Pero pronosticar 2024 con esa innovación tiene una consecuencia matemática: $E[e^T] = \infty$ para cualquier *t*, así que ni la media predictiva ni el CRPS existen. Por eso **el pronóstico de 2024 es condicional a que ningún municipio cambie de régimen**: se persiste el nivel de 2023 con su incertidumbre a posteriori. La §7 muestra numéricamente qué pasa si no se hace así.

```
import warnings
warnings.filterwarnings("ignore")

import time
import numpy as np
from scipy.sparse.csgraph import connected_components
```

```
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap, TwoSlopeNorm
from scipy import stats
import statsmodels.formula.api as smf
import pymc as pm
import arviz as az
from libpysal import weights

from geostats import espacial, rutas

SEMILLA = 42
rng = np.random.default_rng(SEMILLA)

g = gpd.read_parquet(rutas.ATUS_ZMM_LIMPIO_GEO).to_crs(espacial.UTM_ZMM)
N = len(g)
print(f"{N:,} registros | PyMC {pm.__version__}")
```

```
379,294 registros | PyMC 6.3.2
```

Estilo de las gráficas

Misma identidad que los notebooks anteriores.

```
# --- Identidad GeoStats ------------------------------------------------------
AZUL     = "#005991"
ROJO     = "#8B2C1A"
OXIDO    = "#B15E2E"
GRAFITO  = "#2C2C2C"
GRIS     = "#F2F2F2"
BLANCO   = "#FFFFFF"
AZUL_RAMPA = ["#005991", "#1b77b8", "#4195d9"]

SECUENCIAL = LinearSegmentedColormap.from_list("geostats_seq", [BLANCO, AZUL])
DIVERGENTE = LinearSegmentedColormap.from_list("geostats_div", [AZUL, GRIS, ROJO])

TITULAR = ["Montserrat", "Helvetica Neue", "Arial", "DejaVu Sans"]
TEXTO   = ["Cormorant Garamond", "EB Garamond", "Georgia", "DejaVu Serif"]
CIFRAS  = ["Roboto Mono", "Menlo", "DejaVu Sans Mono"]

plt.rcParams.update({
    "figure.dpi": 130, "figure.facecolor": BLANCO, "axes.facecolor": BLANCO,
    "savefig.facecolor": BLANCO, "font.family": "sans-serif",
    "font.sans-serif": TITULAR, "font.serif": TEXTO, "font.monospace": CIFRAS,
    "font.size": 9, "axes.edgecolor": GRIS, "axes.labelcolor": GRAFITO,
    "axes.labelsize": 9, "axes.titlecolor": ROJO, "axes.titlesize": 11.5,
    "axes.titleweight": "bold", "axes.titlelocation": "left", "axes.titlepad": 16,
    "axes.grid": True, "axes.axisbelow": True, "grid.color": GRIS,
```

```
    "grid.linewidth": 1.0, "xtick.color": GRAFITO, "ytick.color": GRAFITO,
    "xtick.labelsize": 8.5, "ytick.labelsize": 8.5, "legend.frameon": False,
    "legend.fontsize": 8.5, "lines.linewidth": 1.8,
})

def pulir(ax, ejes=("top", "right", "left")):
    for lado in ejes:
        ax.spines[lado].set_visible(False)
    ax.tick_params(length=0)
    for etiqueta in ax.get_xticklabels() + ax.get_yticklabels():
        etiqueta.set_fontfamily("monospace")
    return ax

def mapa(ax, titulo=None, escala_km=10):
    ax.set_aspect("equal"); ax.grid(False)
    ax.set_xticks([]); ax.set_yticks([])
    for lado in ax.spines.values():
        lado.set_visible(False)
    if titulo:
        ax.set_title(titulo)
    x0, x1 = ax.get_xlim(); y0, y1 = ax.get_ylim()
    bx = x0 + 0.04 * (x1 - x0); by = y0 + 0.04 * (y1 - y0)
    ax.plot([bx, bx + escala_km * 1000], [by, by], color=GRAFITO, lw=2, solid_capstyle="b
    ax.text(bx, by + 0.012 * (y1 - y0), f"{escala_km} km", fontsize=7.5,
            color=GRAFITO, fontfamily="monospace")
    return ax
```

---

1. El panel anual, y por qué anual

El notebook anterior trabajó sobre celda × mes (161 mil filas). Este trabaja sobre **celda × año** (13 mil), por dos razones que conviene separar:

- **Estadística.** El efecto de mes es común a todas las celdas y al municipio: en `modelo_conteo.qmd` la estacionalidad aportó casi nada sobre un promedio bien elegido (M4 ≈ M0r). Toda la estructura que interesa aquí —el régimen municipal, el perfil de cada celda— vive en el nivel anual. Esto es el supuesto de **separabilidad**: el patrón espacial no cambia de forma dentro del año. Si hiciera falta el detalle mensual, se repone después con un perfil estacional multiplicativo fijo.
- **Cómputo.** NUTS sobre 161 mil observaciones con 2,300 parámetros tarda horas; sobre 13 mil, minutos. La diferencia decide si el modelo se puede iterar o no.

```
H, W = espacial.celdas(g, lado=500)
assert len(H) == 2270 and len(W.islands) == 0

union = gpd.sjoin(g[["geometry", "ANIO", "NOM_MUN"]], H[["hex", "geometry"]],
                  predicate="within", how="inner")
ANIOS = list(range(2019, 2025))
indice = pd.MultiIndex.from_product([H.hex, ANIOS], names=["hex", "anio"])
```

```
p = union.groupby(["hex", "ANIO"]).size().reindex(indice, fill_value=0).rename("y").reset

municipio = union.groupby("hex").NOM_MUN.agg(lambda s: s.mode().iloc[0])
p["municipio"] = p.hex.map(municipio)
MUNICIPIOS = sorted(municipio.unique())
hex_a_i = {h: i for i, h in enumerate(H.hex)}
mun_a_j = {m: j for j, m in enumerate(MUNICIPIOS)}
p["i"] = p.hex.map(hex_a_i)
p["j"] = p.municipio.map(mun_a_j)
p["a"] = p.anio - 2019

tr = p[p.anio <= 2023].reset_index(drop=True)
te = p[p.anio == 2024].reset_index(drop=True)
N_HEX, N_MUN, N_ANIOS = len(H), len(MUNICIPIOS), 5

print(f"Entrenamiento: {len(tr):,} celda-año (2019–2023) | Prueba: {len(te):,} (2024)")
print(f"{N_HEX:,} celdas en {N_MUN} municipios | media anual por celda {tr.y.mean():.1f},
      f"mediana {tr.y.median():.0f}, ceros {(tr.y == 0).mean():.1%}")
```

```
Entrenamiento: 11,350 celda-año (2019–2023) | Prueba: 2,270 (2024)
2,270 celdas en 18 municipios | media anual por celda 27.1, mediana 5, ceros 25.8%
```

---

2. El modelo

La ecuación

Para la celda i, en el municipio $j(i)$, en el año a:

$$Y_{ia} \sim \text{BinomialNegativa}(\mu_{ia},\ \theta)$$

$$\log \mu_{ia} = \beta_0 + m_{j(i),a} + u_i$$

Tres términos, tres niveles:

Término	Qué es	Cómo varía
β_0	Nivel base del área metropolitana	Constante
$m_{j,a}$	**Régimen del municipio** j en el año a	Caminata aleatoria en el tiempo, con colas pesadas
u_i	**Perfil de la celda** i, neto de su municipio	Constante en el tiempo, correlacionado con sus vecinas

El municipio absorbe *cuánto se reporta*; la celda absorbe *dónde, dentro del municipio, se concentra*. Esa separación es la conclusión del notebook anterior convertida en estructura.

El nivel municipal: caminata aleatoria con innovaciones *t*

$$m_{j,2019} = \sigma_m \, z_j, \qquad z \sim \text{NormalSumaCero}(1)$$

$$m_{j,a} = m_{j,a-1} + \sigma_{rw} \, \varepsilon_{j,a}, \qquad \varepsilon \sim t_3$$

La suma cero en el nivel inicial identifica β_0: sin ella, subir todos los municipios y bajar β_0 da la misma verosimilitud. Para 2024 el nivel se **persiste**: $m_{j,2024} = m_{j,2023}$, con la incertidumbre a posteriori de $m_{j,2023}$ pero sin una innovación nueva (ver la advertencia inicial). Las innovaciones son *t* de Student con 3 grados de libertad y no normales por lo que mostraron las series: la mayoría de los municipios se mueve poco de un año a otro y unos pocos saltan (Santa Catarina 77 → 1,795). Una normal tendría que elegir entre un σ_{rw} grande que no describe a nadie o uno pequeño que no admite los saltos; la *t* admite ambos.

El nivel de celda: CAR propio

$$\mathbf{u} \sim \mathcal{N}\big(\mathbf{0},\ \sigma_u^2 \, (D - \alpha A)^{-1}\big)$$

donde A es la matriz de adyacencia de los hexágonos, D la diagonal con el número de vecinos, y $\alpha \in (0, 1)$ la **dependencia espacial**: con $\alpha = 0$ las celdas son independientes; conforme $\alpha \to 1$ cada celda se parece más al promedio de sus vecinas. Es un prior propio (integra a uno), lo que importa en la §3.

Priors

Parámetro	Prior	Por qué
β_0	$\mathcal{N}(\log \bar{y},\ 2)$	Centrado en la media empírica, ancho
σ_m	Semi-normal(1.5)	Los municipios difieren en órdenes de magnitud
σ_{rw}	Semi-normal(0.5)	Un cambio anual típico de menos del 60 %
α	Beta(2, 1)	Inclinado a dependencia positiva; el análisis espacial la midió
σ_u	Semi-normal(1.5)	Ancho; los datos por celda son fuertes
θ	Gamma(2, 0.1)	Media 20; dispersión moderada a escala anual

Supuestos y cómo se verifican

Supuesto	Verificación	Sección
Separabilidad espacio-tiempo	Spearman 0.83 entre 2019 y 2024 (análisis espacial); coherencia de u_i	§1, §6
Régimen municipal como caminata aleatoria	Trayectorias $m_{j,a}$ a posteriori vs. series observadas	§5
Innovaciones de colas pesadas	σ_{rw} con t vs. con normal	§5
Grafo de vecindad completo	`espacial.celdas()` falla si A difiere de la adyacencia geométrica. Sin islas, pero no conexo: componentes en §3	§3
Convergencia del MCMC	$\hat{R}$, ESS, divergencias — sobre las cantidades identificadas	§4
Los priors no dictan el resultado	Reajuste con priors alternativos	§8
Exogeneidad de u_i respecto al municipio	El CAR está centrado en cero; el municipio absorbe el nivel	§6

3. Por qué CAR propio y no BYM2

La propuesta original era BYM2: $u_i = \sigma(\sqrt{1-\phi}\, v_i + \sqrt{\phi}\, s_i)$ con v independiente y s ICAR, y ϕ como "fracción espacial". Se intentó, en las dos parametrizaciones, y **no se identificó**:

Parametrización	Resultado	$\hat{R}$ de ϕ
No centrada ($s \sim$ ICAR(1), luego escalado)	$\phi \to 0.0001$: la parte espacial desaparece	2.06
Centrada ($s \sim$ ICAR(σ_s))	$\sigma_s \to 13$, $\phi \to 0.9998$: la parte independiente desaparece	1.65

El motivo es de estructura, no de *tuning*. Con cinco años de conteos por celda, los datos fijan cada u_i casi por completo; la descomposición en "parte espacial" y "parte independiente" solo la sostienen los priors, y el muestreador cae en el modo degenerado que le quede más cerca. El Moran de la tasa por celda tras quitar la media municipal es 0.41 — la estructura espacial *existe*; lo que no existe es información para repartirla entre dos componentes.

El CAR propio tiene una sola varianza y un solo parámetro de dependencia. Es un prior propio, está identificado con datos fuertes, y α responde la misma pregunta que ϕ ("¿cuánto se parecen las

vecinas?") de forma directamente estimable. Es la elección que recomienda la literatura cuando BYM no converge (Banerjee, Carlin & Gelfand 2014, §6.4).

```
Wb = weights.W(W.neighbors)
Wb.transform = "b"
A = Wb.full()[0]
assert (A == A.T).all(), "la adyacencia debe ser simétrica"
assert (A.sum(1) == np.array([Wb.cardinalities[i] for i in range(N_HEX)])).all()
n_componentes, etiquetas = connected_components(A, directed=False)
mayor_componente = np.bincount(etiquetas).max()
print(f"Adyacencia: {N_HEX:,} × {N_HEX:,}, {int(A.sum() / 2):,} aristas, "
      f"{A.sum(1).mean():.2f} vecinos por celda")
print(f"Componentes conexas: {n_componentes}; la mayor, {mayor_componente:,} celdas "
      f"({mayor_componente / N_HEX:.1%})")
```

```
Adyacencia: 2,270 × 2,270, 5,325 aristas, 4.69 vecinos por celda
Componentes conexas: 35; la mayor, 2,126 celdas (93.7%)
```

El grafo no es conexo. $D - \alpha A$ sigue siendo definida positiva para $\alpha < 1$, porque el CAR propio no requiere conexidad, pero el encogimiento hacia las vecinas opera solo dentro de cada componente. Un ICAR, en cambio, exigiría una restricción de suma cero por componente.

---

4. Ajuste y convergencia

```
def construir(alpha_prior=(2, 1), sigma_u_escala=1.5, theta_prior=(2, 0.1)):
    with pm.Model() as modelo:
        b0 = pm.Normal("b0", np.log(tr.y.mean() + 1), 2)

        s_m = pm.HalfNormal("s_m", 1.5)
        s_rw = pm.HalfNormal("s_rw", 0.5)
        z0 = pm.ZeroSumNormal("z0", sigma=1, shape=N_MUN)
        eps = pm.StudentT("eps", nu=3, mu=0, sigma=1, shape=(N_MUN, N_ANIOS - 1))
        m0 = (s_m * z0)[:, None]
        m = pm.Deterministic("m", pm.math.concatenate([m0, m0 + pm.math.cumsum(s_rw * eps

        alpha = pm.Beta("alpha", *alpha_prior)
        s_u = pm.HalfNormal("s_u", sigma_u_escala)
        u = pm.CAR("u", mu=np.zeros(N_HEX), W=A, alpha=alpha, tau=1 / s_u**2, shape=N_HEX

        theta = pm.Gamma("theta", *theta_prior)
        log_mu = pm.Deterministic("log_mu", b0 + m[tr.j.values, tr.a.values] + u[tr.i.val
        pm.NegativeBinomial("y", mu=pm.math.exp(log_mu), alpha=theta, observed=tr.y.value
    return modelo

CALENTAMIENTO, MUESTRAS = 3000, 4000

def muestrear(modelo, draws=MUESTRAS, tune=CALENTAMIENTO, chains=4):
```

```
    inicio = time.time()
    idata = pm.sample(draws=draws, tune=tune, chains=chains, nuts_sampler="nutpie",
                      random_seed=SEMILLA, progressbar=False, model=modelo)
    idata.attrs["segundos"] = time.time() - inicio
    return idata

modelo = construir()
idata = muestrear(modelo)
print(f"4 cadenas × ({CALENTAMIENTO:,} calentamiento + {MUESTRAS:,} muestras) "
      f"en {idata.attrs['segundos'] / 60:.1f} min")
```

```
NUTS[nutpie]: [b0, s_m, s_rw, z0, eps, alpha, s_u, u, theta]

4 cadenas × (3,000 calentamiento + 4,000 muestras) en 9.2 min
```

Diagnósticos

Tres criterios, todos necesarios: $\hat{R} < 1.01$ (las cadenas coinciden), tamaño efectivo de muestra suficiente (las muestras no están demasiado correlacionadas) y cero transiciones divergentes (el integrador no se rompió en ninguna región).

El criterio se exige a las cantidades **identificadas**, los $\log \mu$. En los hiperparámetros se reporta $\hat{R}$ y ESS sin exigir el umbral: β_0 por separado no está identificado del todo (compite con la media de u, que el CAR solo ata débilmente a cero), y α comparte información con σ_u cuando la dependencia se acerca a 1.

```
HIPER = ["b0", "s_m", "s_rw", "alpha", "s_u", "theta"]
resumen = az.summary(idata, var_names=HIPER)[["mean", "sd", "ess_bulk", "r_hat"]]
divergencias = int(idata.sample_stats.diverging.sum())
rhat_logmu = float(az.rhat(idata, var_names=["log_mu"]).log_mu.max())
ess_logmu = float(az.ess(idata, var_names=["log_mu"]).log_mu.min())

print(f"Divergencias: {divergencias}")
print(f"log μ — R̂ máximo {rhat_logmu:.3f}, ESS mínimo {ess_logmu:,.0f} (sobre {len(tr):,}
resumen.round(3)
```

```
Divergencias: 0
log μ — R̂ máximo 1.002, ESS mínimo 4,289 (sobre 11,350 cantidades)
```

	mean	sd	ess_bulk	r_hat
b0	1.5	0.12	149	1.02
s_m	1.17	0.23	78	1.02
s_rw	0.289	0.036	259	1.01
alpha	0.919	0.0146	219	1.02
s_u	2.606	0.0472	2357	1.00
theta	12.215	0.447	10692	1.00

Cero divergencias; el $\hat{R}$ máximo de las 11,350 $\log \mu$ es 1.002 y su ESS mínimo supera cuatro mil. σ_u y θ mezclan bien. α, β_0 y σ_m quedan en $\hat{R} = 1.02$, con ESS de 219, 149 y 78, y σ_{rw} en 1.01: no cumplen el umbral. Duplicar calentamiento y muestras (de 1,000 + 2,000 a 3,000 + 4,000 por cadena) bajó el $\hat{R}$ de α de 1.04 a 1.02 sin llegar a 1.01, porque la mezcla lenta es estructural. La estimación de α es estable: 0.917 con la corrida corta, 0.919 con esta y 0.921 con priors alternativos (§8). Ninguno de esos parámetros se usa solo en una conclusión; las conclusiones descansan en $\log \mu$ y en las medianas de $m_{j,a}$ y u_i.

---

5. Lo que el modelo aprendió

```
post = az.extract(idata)
print(f"α (dependencia espacial CAR): {post.alpha.mean():.3f}   [{np.percentile(post.alph
print(f"θ (dispersión NB):            {post.theta.mean():.1f}")
print(f"σ_rw (paso anual municipal):  {post.s_rw.mean():.3f}  → un cambio típico de ×{np.
print(f"σ_u (dispersión entre celdas): {post.s_u.mean():.2f}")
```

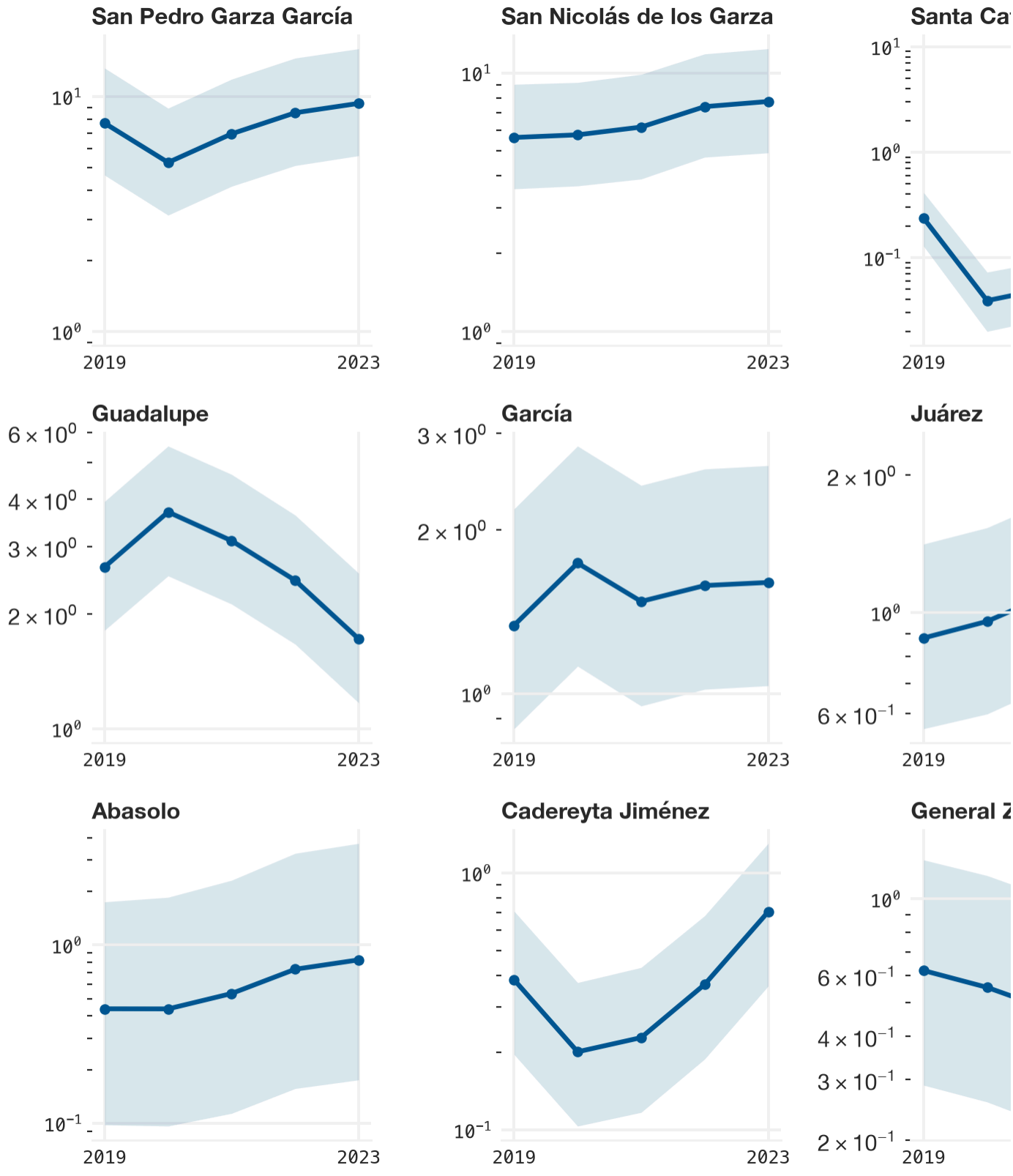
```
α (dependencia espacial CAR): 0.919   [0.890, 0.945]
θ (dispersión NB):            12.2
σ_rw (paso anual municipal):  0.289  → un cambio típico de ×1.33 al año
σ_u (dispersión entre celdas): 2.61
```

- $\alpha = 0.92$. Dependencia espacial fuerte; su intervalo es angosto, pero la cadena mezcla lento ($\hat{R}$ 1.02). El perfil de una celda se parece mucho al de sus vecinas, incluso después de quitar el municipio. Es el I de Moran intra-municipal de 0.41 visto desde dentro del modelo.
- $\theta = 12$ a escala anual, contra 5.5 a escala mensual. Al agregar doce meses la sobredispersión relativa baja, como debe: parte del ruido mensual se promedia.
- $\sigma_{rw} = 0.29$ con innovaciones *t*. Con innovaciones normales salía 0.61: los saltos de Santa Catarina y Salinas Victoria inflaban la escala al doble. La *t* deja que el municipio típico cambie ~30 % al año y reserva las colas para los que saltan.

Trayectorias municipales

```
m_post = post.m.values          # (mun, año, draws)
q = np.percentile(np.exp(m_post), [5, 50, 95], axis=2)
orden = np.argsort(-q[1, :, -1])
fig, ejes = plt.subplots(3, 6, figsize=(13, 6.4), sharey=False)
for ax, j in zip(ejes.flat, orden):
    ax.fill_between(ANIOS[:-1], q[0, j], q[2, j], color=AZUL, alpha=0.15, linewidth=0)
    ax.plot(ANIOS[:-1], q[1, j], color=AZUL, marker="o", ms=3)
    ax.axhline(1, color=GRIS, lw=1)
    ax.set_yscale("log")
    ax.set_title(MUNICIPIOS[j], fontsize=8.5, color=GRAFITO, pad=4)
    ax.set_xticks([2019, 2023]); ax.tick_params(labelsize=7)
```

```
    pulir(ax)
plt.tight_layout(); plt.show()
```



Efecto municipal m_{j,a} a posteriori (mediana e intervalo del 90 %), en escala multiplicativa respecto a la media metropolitana. Cada panel es un régimen de reporte.

Las trayectorias reproducen las series del notebook anterior, ahora como parámetros con incertidumbre: Santa Catarina sube dos órdenes de magnitud entre 2021 y 2022; Guadalupe cae; la periferia (Pesquería, Santiago, Cadereyta) sube sostenidamente. El intervalo es angosto en los municipios grandes y ancho en los que tienen pocas celdas: el modelo sabe cuánto sabe.

---

6. El perfil espacial de las celdas

u_i es el producto principal de este modelo: **cuánto más o menos riesgo tiene cada celda que el promedio de su municipio**, estimado con la información de la celda *y* de sus vecinas, y con la incertidumbre que corresponde a cuántos accidentes tiene.

Shrinkage

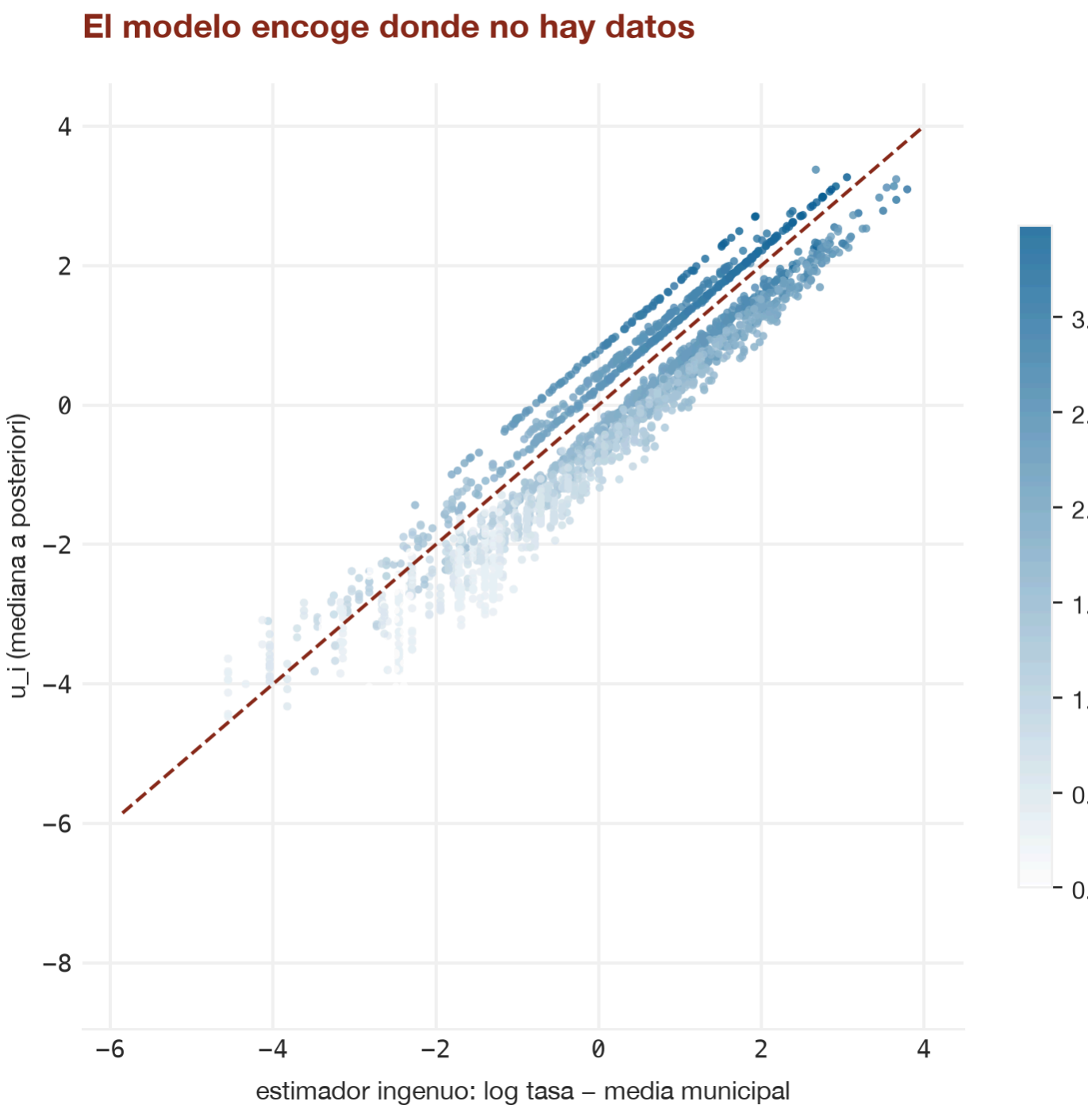
Se compara u_i con la estimación ingenua —el logaritmo de la tasa de la celda menos la de su municipio— para ver cuánto encoge el modelo y a quién.

```
u_post = post.u.values                          # (hex, draws)
H["u_med"] = np.median(u_post, axis=1)
H["u_sd"] = u_post.std(axis=1)
H["municipio"] = H.hex.map(municipio)
tasa = tr.groupby("hex").y.mean().reindex(H.hex).values
H["log_tasa"] = np.log(tasa + 0.1)   # medio conteo en cinco años: corrección de Anscombe
H["ingenuo"] = H.log_tasa - H.groupby("municipio").log_tasa.transform("mean")
H["n_train"] = tr.groupby("hex").y.sum().reindex(H.hex).values

pocos = H.n_train <= 5
print(f"Celdas con ≤ 5 accidentes en 2019–2023: {pocos.sum():,} ({pocos.mean():.0%})")
print(f"  desviación del estimador ingenuo: {H.loc[pocos, 'ingenuo'].std():.2f}   |   de
print(f"  incertidumbre media de u_i:       {H.loc[pocos, 'u_sd'].mean():.2f}   |   en ce
```

```
Celdas con ≤ 5 accidentes en 2019–2023: 643 (28%)
  desviación del estimador ingenuo: 1.10   |   de u_i: 0.94
  incertidumbre media de u_i:       0.79   |   en celdas con > 100: 0.28
```

```
fig, ax = plt.subplots(figsize=(6, 5.6))
sc = ax.scatter(H.ingenuo, H.u_med, c=np.log10(H.n_train + 1), cmap=SECUENCIAL, s=8, line
lim = [H.ingenuo.min() - 0.2, H.ingenuo.max() + 0.2]
ax.plot(lim, lim, color=ROJO, lw=1.2, ls="--")
ax.set_xlabel("estimador ingenuo: log tasa − media municipal")
ax.set_ylabel("u_i (mediana a posteriori)")
ax.set_title("El modelo encoge donde no hay datos")
plt.colorbar(sc, ax=ax, shrink=0.7, label="log₁₀ accidentes 2019–23")
pulir(ax); plt.tight_layout(); plt.show()
```



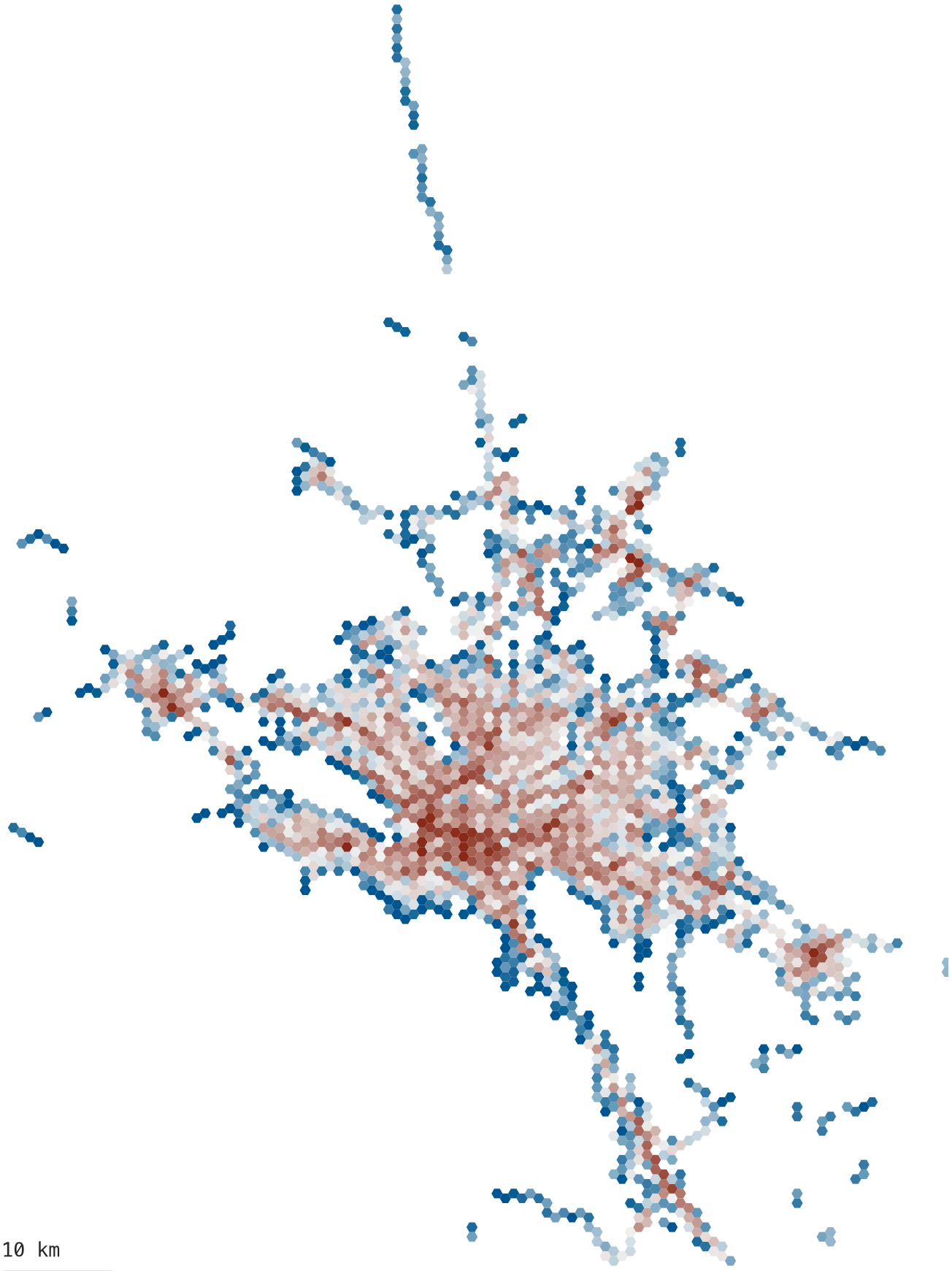
Estimador ingenuo contra u_i a posteriori. Cada punto es una celda; el color, cuántos accidentes tuvo en entrenamiento. Los puntos con pocos datos se acercan a la diagonal desde abajo: el modelo los encoge hacia su municipio y sus vecinas.

En las 643 celdas con cinco accidentes o menos en cinco años, la dispersión de u_i es un 15 % menor que la del estimador ingenuo: el encogimiento del *punto* es modesto, porque $\sigma_u = 2.6$ es un prior ancho y el modelo deja hablar a los datos aun cuando son pocos. Donde el *shrinkage* se ve de verdad es en la **incertidumbre**: 0.79 en esas celdas contra 0.28 en las que tienen más de cien accidentes, casi tres veces más. El modelo no mueve mucho la estimación de una celda pobre en datos; lo que hace es decir, con el ancho de su intervalo, cuánto no sabe de ella. **Ese es el *shrinkage* proporcional a la ignorancia** que motivó el enfoque jerárquico, y para decidir es el que importa: dos celdas con el mismo u_i y distinta incertidumbre no merecen la misma intervención.

El mapa neto del régimen municipal

```
fig, ax = plt.subplots(figsize=(8.5, 8))
norma = TwoSlopeNorm(vmin=-3, vcenter=0, vmax=3)
H.plot(column="u_med", cmap=DIVERGENTE, norm=norma, ax=ax, linewidth=0, legend=True,
       legend_kwds={"shrink": 0.5, "label": "u_i (log; ±3 ≈ ×20 / ÷20)", "pad": 0.01})
mapa(ax, "Perfil espacial neto del municipio")
plt.tight_layout(); plt.show()
```

Perfil espacial neto del municipio



u_i: el riesgo relativo de cada celda respecto a su municipio, en escala multiplicativa. Este mapa no depende de cuánto reporta cada municipio.

Es el mapa que el análisis espacial no podía dar: ahí el conteo de una celda mezclaba *dónde está* con *cuánto reporta su municipio*. Aquí el municipio se fue a $m_{j,a}$ y lo que queda son los corredores. Nótese que Guadalupe y San Nicolás ya no aparecen como bloques: sus celdas se comparan con su propio promedio, y dentro de cada uno hay avenidas rojas y colonias azules como en Monterrey.

---

7. Validación en 2024

El pronóstico para 2024 es una simulación del modelo: para cada muestra posterior se toma el nivel municipal de 2023, se combina con u_i, y se simula un conteo Binomial Negativo. De ahí salen mediana, intervalos y CRPS. La incertidumbre incluye la de todos los parámetros y la dispersión NB; **no incluye un cambio de régimen municipal en 2024**, por la razón matemática de la advertencia inicial.

```
m_2023 = m_post[:, -1, :]                                  # (mun, draws)
theta_post = post.theta.values
y24 = te.y.values

def pronosticar(m_2024):
    lam = np.exp(post.b0.values[None, :] + m_2024[te.j.values, :] + u_post[te.i.values, :
    y_sim = rng.negative_binomial(theta_post[None, :], theta_post[None, :] / (theta_post[
    return lam, y_sim

# Persistencia: el nivel de 2023 con su incertidumbre a posteriori.
lam, y_sim = pronosticar(m_2023)
pred_med = np.median(lam, axis=1)
pred_media = lam.mean(axis=1)

# Lo que pasaría con una innovación t más: la media deja de existir.
lam_t, _ = pronosticar(m_2023 + rng.standard_t(3, size=m_2023.shape) * post.s_rw.values[N

print(f"Total 2024 observado: {y24.sum():,}")
print(f"  persistencia — suma de medianas {pred_med.sum():,.0f} (cociente {pred_med.sum()
      f"suma de medias {pred_media.sum():,.0f} (cociente {pred_media.sum() / y24.sum():.3
print(f"  con innovación t — suma de medianas {np.median(lam_t, axis=1).sum():,.0f}, "
      f"suma de medias {lam_t.mean(axis=1).sum():,.0f}  ← {lam_t.mean(axis=1).sum() / y24
```

```
Total 2024 observado: 71,295
  persistencia — suma de medianas 70,810 (cociente 0.993), suma de medias 71,826
(cociente 1.007)
  con innovación t — suma de medianas 71,078, suma de medias 2,762,662  ← 39 veces el
total; no existe
```

Con persistencia, la suma de medianas calibra el total de 2024 al 1 % y la de medias también existe y está cerca. Con una innovación *t* más, la mediana sigue siendo razonable pero la media es un número sin sentido: unas pocas muestras extremas de la cola la dominan, y el valor cambia con la semilla. Es la advertencia del inicio hecha visible, y la razón de pronosticar por persistencia.

Contra los *baselines*

Los mismos tres competidores del notebook anterior, ahora a escala anual: la media de 2019-2023 (M0), la de 2022-2023 (M0r) y la Binomial Negativa con historia reciente (M4). Sus intervalos son Poisson (M0, M0r) o NB (M4); el del jerárquico es el ensamble.

```
def crps_ensamble(y, X):
    """CRPS de un ensamble de simulaciones: E|X-y| - ½ E|X-X'|, por la fórmula ordenada."
    X = np.sort(X, axis=1)
    n = X.shape[1]
    t1 = np.mean(np.abs(X - y[:, None]), axis=1)
    w = 2 * np.arange(1, n + 1) - n - 1
    t2 = 2 * np.sum(w * X, axis=1) / (n * (n - 1))
    return t1 - 0.5 * t2

def pai(pred, y, f):
    k = max(1, int(len(pred) * f))
    top = np.argsort(-pred)[:k]
    return (y[top].sum() / y.sum()) / f

def evaluar(nombre, punto, sim):
    lo, hi = np.percentile(sim, [5, 95], axis=1)
    return {"modelo": nombre, "MAE": np.mean(np.abs(y24 - punto)),
            "CRPS": crps_ensamble(y24, sim).mean(),
            "cobertura 90 %": np.mean((y24 >= lo) & (y24 <= hi)),
            "PAI 5 %": pai(punto, y24, 0.05), "PAI 10 %": pai(punto, y24, 0.10)}

S = lam.shape[1]
h = tr.groupby("hex").y.mean().reindex(H.hex).values + 0.1
h_rec = tr[tr.anio >= 2022].groupby("hex").y.mean().reindex(H.hex).values + 0.25

# M4 anual: NB2 con log h_rec y periodo, ajustada en 2019-2023
tr_glm = tr.assign(logh_rec=np.log(h_rec[tr.i.values]),
                   periodo=np.where(tr.anio == 2019, "pre", np.where(tr.anio <= 2021, "pa
te_glm = te.assign(logh_rec=np.log(h_rec[te.i.values]), periodo="georef")
m4 = smf.negativebinomial("y ~ C(periodo) + logh_rec", data=tr_glm).fit(disp=0, maxiter=2
theta4 = 1 / m4.params["alpha"]
mu4 = m4.predict(te_glm).values

resultados = pd.DataFrame([
    evaluar("M0  media 2019-23", h, rng.poisson(h[:, None], size=(N_HEX, S))),
    evaluar("M0r media 2022-23", h_rec, rng.poisson(h_rec[:, None], size=(N_HEX, S))),
    evaluar("M4  NB2, historia reciente", mu4, rng.negative_binomial(theta4, theta4 / (th
    evaluar("M6  jerárquico CAR + municipio", pred_med, y_sim),
]).set_index("modelo")
```

```
resultados.style.format({"MAE": "{:.2f}", "CRPS": "{:.3f}", "cobertura 90 %": "{:.1%}",
                         "PAI 5 %": "{:.2f}", "PAI 10 %": "{:.2f}"})
```

	MAE	CRPS	cobertura 90 %	PAI 5 %	PAI 10 %
modelo					
M0 media 2019-23	9.49	7.955	63.6%	8.44	5.85
M0r media 2022-23	8.06	6.493	68.6%	8.42	5.96
M4 NB2, historia reciente	8.07	6.852	90.0%	8.42	5.96
M6 jerárquico CAR + municipio	8.10	6.097	87.2%	8.44	5.96

Lo que dice la tabla:

- **El jerárquico empata con M0r en MAE y lo supera en CRPS** (6.10 contra 6.49, un 6 %). El empate en MAE dice que la *mediana* de ambos apunta al mismo sitio: lo predecible de 2024 es "el nivel reciente de cada municipio por el perfil estable de cada celda", y un promedio bien elegido ya lo captura. La ventaja en CRPS dice que la *distribución* del jerárquico es mejor: acierta igual en el centro y describe bien lo que hay alrededor.
- **La diferencia está en la incertidumbre.** El intervalo del 90 % de M0r cubre el 69 % de lo observado: es un Poisson que ignora la sobredispersión y el régimen, y se queda corto. El del jerárquico cubre el 87 %, cerca del nominal, sin incluir siquiera el riesgo de cambio de régimen. Un intervalo que no cubre lo que dice cubrir no sirve para decidir con él.
- **El PAI no se mueve.** Como en el notebook anterior, ninguna especificación cambia el orden de las celdas de forma material; el 5 % superior ya lo daba el histórico. El *shrinkage* reordena la cola, no la cabeza — que es donde se decide.

Dónde sigue fallando

```
por_mun = (pd.DataFrame({"y": y24, "pred": pred_med, "municipio": te.municipio.values})
           .groupby("municipio").sum())
por_mun["cociente"] = por_mun.y / por_mun.pred
por_mun.sort_values("cociente").style.format({"y": "{:,.0f}", "pred": "{:,.0f}", "cocient
```

	y	pred	cociente
municipio			
Abasolo	3	10	0.30
Santa Catarina	2,747	3,637	0.76
Guadalupe	2,770	3,566	0.78
San Nicolás de los Garza	9,226	11,357	0.81
General Escobedo	3,422	4,158	0.82
Apodaca	3,618	4,121	0.88
Hidalgo	92	104	0.89

	y	pred	cociente
municipio			
García	2,394	2,470	0.97
San Pedro Garza García	6,878	6,537	1.05
Juárez	1,743	1,631	1.07
Monterrey	33,116	29,802	1.11
Cadereyta Jiménez	630	560	1.13
Pesquería	1,092	867	1.26
Santiago	1,153	780	1.48
General Zuazua	695	441	1.58
Ciénega de Flores	713	354	2.01
Salinas Victoria	819	342	2.39
El Carmen	184	72	2.56

El patrón es el del notebook anterior, atenuado: Guadalupe sigue cayendo más de lo que su caminata aleatoria esperaba, y la periferia (Ciénega, Salinas Victoria, El Carmen) sigue creciendo más. Se probó añadir una **deriva por municipio** (tendencia local) para capturar el crecimiento periférico: el modelo la encogió a cero ($\sigma_d = 0.07$, factores entre 0.98 y 1.05), introdujo 46 divergencias y no mejoró el CRPS. Con cinco años y saltos de régimen, los datos no distinguen una tendencia de una racha de innovaciones.

---

8. Sensibilidad a los priors

Si las conclusiones dependieran de los priors, no serían de los datos. Se reajusta con priors distintos en los tres parámetros donde eso podría importar: α uniforme (sin inclinación a dependencia positiva), σ_u el doble de ancho y θ con media 5 en lugar de 20.

```
modelo_alt = construir(alpha_prior=(1, 1), sigma_u_escala=3.0, theta_prior=(2, 0.4))
idata_alt = muestrear(modelo_alt, draws=1000)
post_alt = az.extract(idata_alt)

lam_alt = np.exp(post_alt.b0.values[None, :] + post_alt.m.values[:, -1, :][te.j.values, :
th_alt = post_alt.theta.values
y_sim_alt = rng.negative_binomial(th_alt[None, :], th_alt[None, :] / (th_alt[None, :] + l

comparacion = pd.DataFrame({
    "priors del modelo": [float(post.alpha.mean()), float(post.theta.mean()), float(post.
                          crps_ensamble(y24, y_sim).mean(), pai(pred_med, y24, 0.05)],
    "priors alternativos": [float(post_alt.alpha.mean()), float(post_alt.theta.mean()), f
                            crps_ensamble(y24, y_sim_alt).mean(), pai(np.median(lam_alt,
}, index=["α", "θ", "σ_rw", "σ_u", "CRPS 2024", "PAI 5 %"])
print(f"Divergencias con priors alternativos: {int(idata_alt.sample_stats.diverging.sum()
```

```
      f"R̂ máx log μ: {float(az.rhat(idata_alt, var_names=['log_mu']).log_mu.max()):.3f}")
comparacion.round(3)
```

```
NUTS[nutpie]: [b0, s_m, s_rw, z0, eps, alpha, s_u, u, theta]

Divergencias con priors alternativos: 2 | R̂ máx log μ: 1.009
```

	priors del modelo	priors alternativos
α	0.919	0.921
θ	12.215	12.146
σ_rw	0.289	0.287
σ_u	2.606	2.606
CRPS 2024	6.097	6.101
PAI 5 %	8.439	8.439

θ cambia 0.6 % (12.215 a 12.146); α, σ_{rw} y σ_u, a lo sumo 0.002, y el CRPS, 0.004. El PAI no cambia. El reajuste deja 2 transiciones divergentes y un $\hat{R}$ máximo de 1.009 en $\log \mu$; las divergencias se reportan, pero no mueven las cifras. Los datos dominan: con 11 mil celda-año y cinco años por celda, el prior de α, σ_u o θ no tiene dónde influir.

Conclusiones

Sobre la estructura. El modelo separa lo que el notebook anterior encontró mezclado: un régimen de reporte por municipio que cambia en el tiempo ($m_{j,a}$, con $\sigma_{rw} = 0.29$ y colas pesadas) y un perfil espacial por celda estable y fuertemente correlacionado con sus vecinas ($\alpha = 0.92$). Los dos se estiman sin divergencias, con $\hat{R}$ de 1.002 en las cantidades identificadas, y son insensibles a los priors. Los hiperparámetros α, β_0 y σ_m mezclan lento ($\hat{R}$ 1.02).

Sobre BYM2. No se identifica con estos datos: en dos parametrizaciones el muestreador colapsó a $\phi = 0$ y a $\phi = 1$. No es que falte estructura espacial (Moran intra-municipal 0.41) sino que sobra información por celda para que los priors decidan cómo repartirla. El CAR propio la estima sin ese problema.

Sobre el *shrinkage*. En las 643 celdas con cinco accidentes o menos en cinco años, el modelo encoge poco la estimación puntual (15 %) pero casi triplica la incertidumbre respecto a las celdas con más de cien. El prior ancho deja hablar a los datos; la incertidumbre dice cuánto no saben.

Sobre la predicción. El jerárquico calibra el total de 2024 al 1 %, empata con el *baseline* reciente en MAE y lo supera en CRPS (6.10 contra 6.49). La mediana no mejora porque lo predecible ya lo capturaba un promedio bien elegido; la distribución sí mejora, y sus intervalos cubren el 87 % contra el 69 % del *baseline*. Lo que ningún modelo captura —los cambios de régimen municipal— no está en el pasado.

Sobre el PAI. Ninguna especificación lo mueve de 8.4. El *dónde* está decidido por el historial y ningún término temporal ni espacial lo cambia. Para moverlo hacen falta covariables que varíen por celda y que el historial no contenga: exposición vial, uso de suelo, población. Eso es el paso siguiente y requiere datos externos.

El producto de este notebook es el mapa de u_i: el riesgo relativo de cada celda neto de cuánto reporta su municipio. Es el primer mapa de la serie que compara Guadalupe con Guadalupe y Monterrey con Monterrey, y por eso es el que sirve para decidir dónde intervenir dentro de cada uno.

Advertencia metodológica

Dos limitaciones que hay que arrastrar a cualquier uso de estos resultados. La primera es la del inicio: el pronóstico es condicional a que ningún municipio cambie de régimen en 2024, porque incluir esa posibilidad con la innovación *t* del modelo deja al pronóstico sin media ni CRPS. Los intervalos son, por tanto, un piso de la incertidumbre real. La segunda es la del notebook anterior, que este modelo formaliza pero no resuelve: $m_{j,a}$ es *reporte*, no siniestralidad. Un municipio que deja de capturar accidentes aparece aquí como un municipio donde bajó el riesgo. El modelo no puede distinguirlos; quien lo use, sí debe.